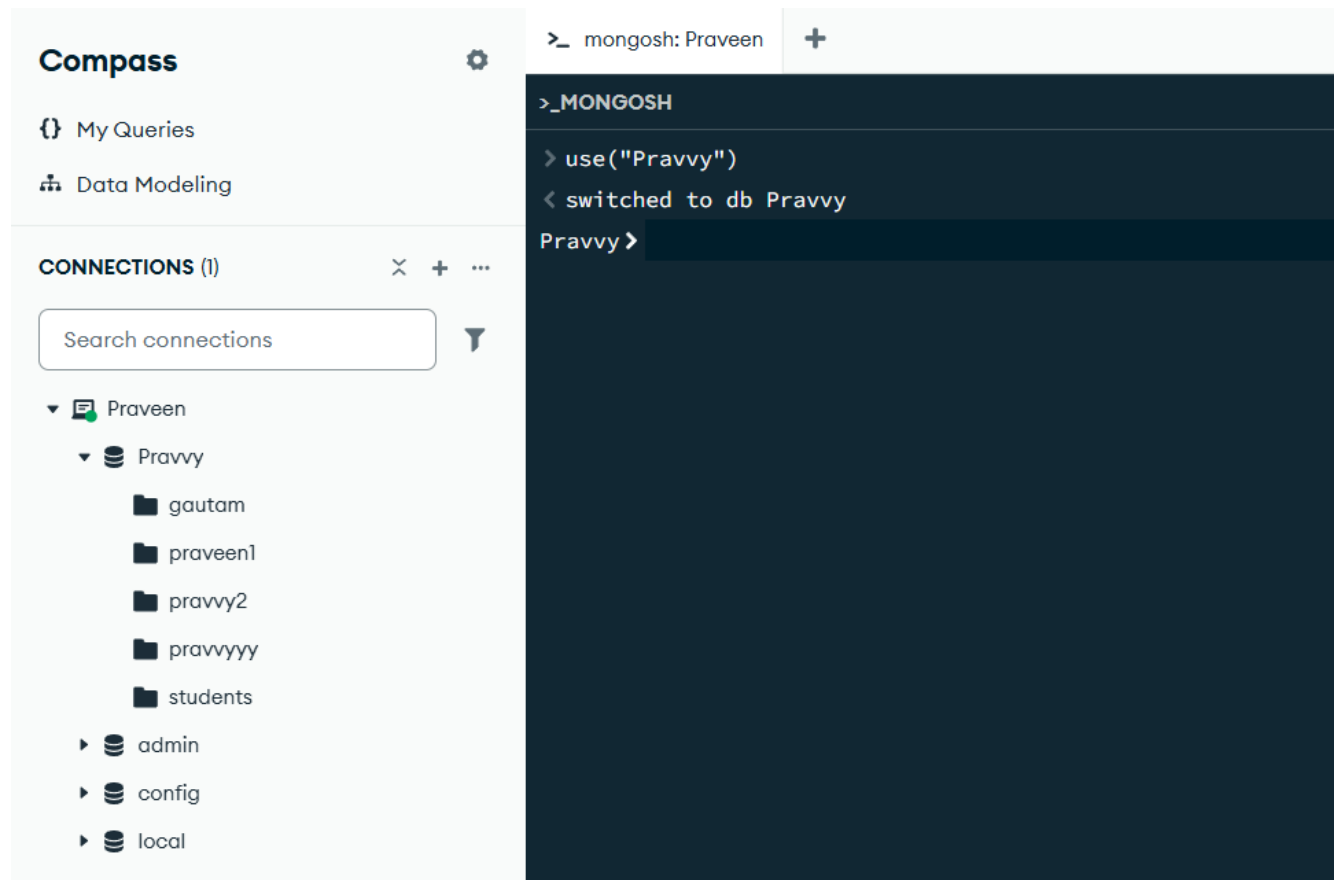


1. CREATE DATABASE

MongoDB Query:

use Students

Explanation: Creates or switches to the Student database.

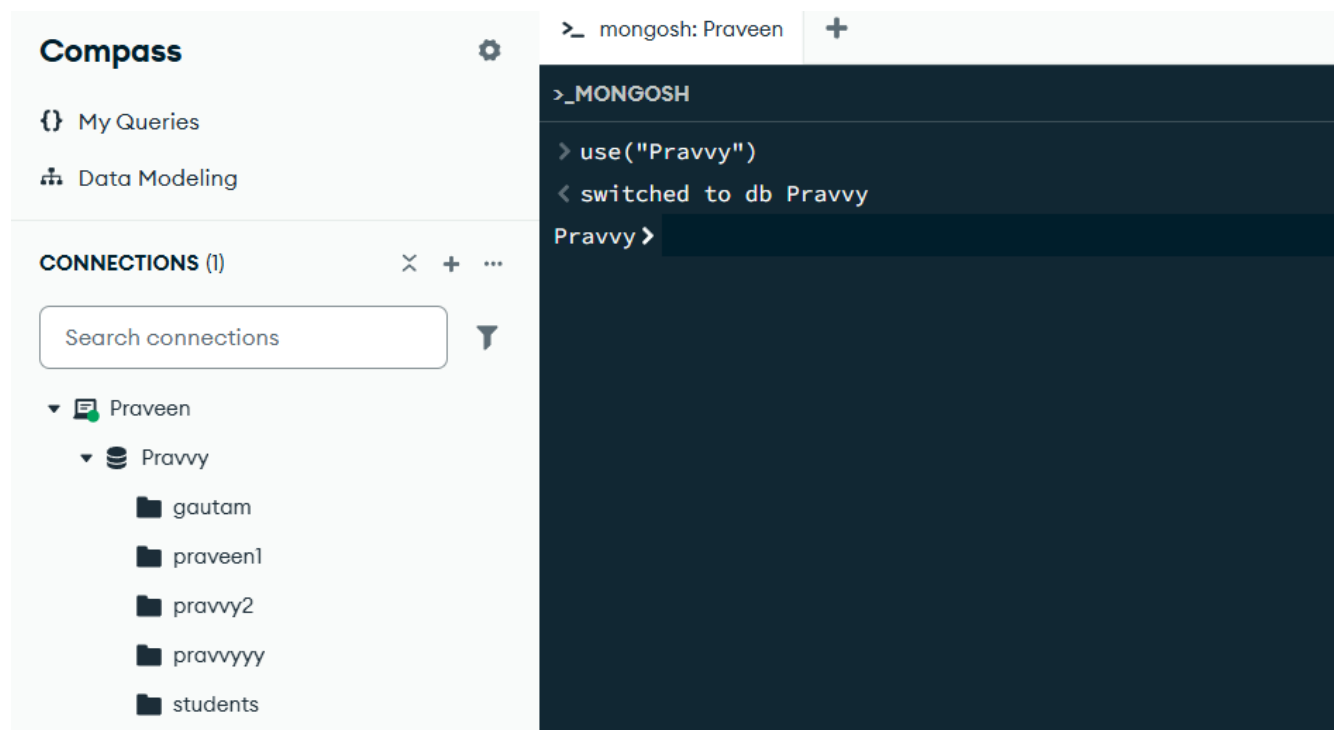


2. CREATE COLLECTION

MongoDB Query:

```
db.createCollection("students")
```

Explanation: Creates a collection named students.



3. INSERT 50 STUDENTS

MongoDB Query:

```
db.Students.insertMany([  
{roll:101,name:"Rahul",age:20,marks:85,city:"Lucknow"},
```

{roll:102,name:"Priya",age:21,marks:90,city:"Kanpur"}.
{roll:103,name:"Amit",age:19,marks:75,city:"Varanasi"}.
{roll:104,name:"Neha",age:22,marks:88,city:"Prayagraj"}.
{roll:105,name:"Rohan",age:20,marks:92,city:"Agra"}.
{roll:106,name:"Anjali",age:21,marks:81,city:"Lucknow"}.
{roll:107,name:"Vivek",age:23,marks:69,city:"Ayodhya"}.
{roll:108,name:"Sneha",age:20,marks:95,city:"Varanasi"}.
{roll:109,name:"Karan",age:22,marks:78,city:"Kanpur"}.
{roll:110,name:"Pooja",age:19,marks:87,city:"Lucknow"}.
{roll:111,name:"Arjun",age:21,marks:91,city:"Agra"}.
{roll:112,name:"Simran",age:20,marks:73,city:"Varanasi"}.
{roll:113,name:"Aditya",age:22,marks:84,city:"Prayagraj"}.
{roll:114,name:"Kavya",age:21,marks:89,city:"Lucknow"}.
{roll:115,name:"Ravi",age:23,marks:76,city:"Kanpur"}.
{roll:116,name:"Nisha",age:20,marks:93,city:"Agra"}.
{roll:117,name:"Sahil",age:19,marks:68,city:"Ayodhya"}.
{roll:118,name:"Isha",age:22,marks:96,city:"Varanasi"}.
{roll:119,name:"Varun",age:21,marks:82,city:"Lucknow"}.
{roll:120,name:"Riya",age:20,marks:86,city:"Kanpur"}.
{roll:121,name:"Manish",age:22,marks:74,city:"Agra"}.
{roll:122,name:"Ayesha",age:21,marks:90,city:"Varanasi"}.
{roll:123,name:"Deepak",age:23,marks:79,city:"Prayagraj"}.
{roll:124,name:"Shreya",age:19,marks:94,city:"Lucknow"}.
{roll:125,name:"Mohit",age:20,marks:83,city:"Kanpur"}.
{roll:126,name:"Tanvi",age:21,marks:77,city:"Agra"}.
{roll:127,name:"Akash",age:22,marks:85,city:"Varanasi"}.
{roll:128,name:"Muskan",age:20,marks:98,city:"Lucknow"}.

{roll:129,name:"Yash",age:19,marks:72,city:"Ayodhya"},
{roll:130,name:"Divya",age:23,marks:89,city:"Prayagraj"},
{roll:131,name:"Raj",age:20,marks:45,city:"Lucknow"},
{roll:132,name:"Komal",age:21,marks:62,city:"Kanpur"},
{roll:133,name:"Suresh",age:19,marks:38,city:"Varanasi"},
{roll:134,name:"Payal",age:22,marks:71,city:"Prayagraj"},
{roll:135,name:"Nitin",age:20,marks:55,city:"Agra"},
{roll:136,name:"Shalini",age:21,marks:48,city:"Lucknow"},
{roll:137,name:"Gaurav",age:23,marks:66,city:"Ayodhya"},
{roll:138,name:"Ritika",age:20,marks:35,city:"Varanasi"},
{roll:139,name:"Harsh",age:22,marks:59,city:"Kanpur"},
{roll:140,name:"Mansi",age:19,marks:43,city:"Lucknow"},
{roll:141,name:"Abhishek",age:21,marks:81,city:"Agra"},
{roll:142,name:"Sakshi",age:20,marks:52,city:"Varanasi"},
{roll:143,name:"Rohit",age:22,marks:68,city:"Prayagraj"},
{roll:144,name:"Pallavi",age:21,marks:47,city:"Lucknow"},
{roll:145,name:"Kunal",age:23,marks:73,city:"Kanpur"},
{roll:146,name:"Rashmi",age:20,marks:29,city:"Agra"},
{roll:147,name:"Dev",age:19,marks:57,city:"Ayodhya"},
{roll:148,name:"Megha",age:22,marks:64,city:"Varanasi"},
{roll:149,name:"Varsha",age:21,marks:41,city:"Lucknow"},
{roll:150,name:"Sameer",age:20,marks:76,city:"Prayagraj"}])

Explanation: Inserts multiple student documents into the collection.

>_MONGOSH

> use("Pravvy")

< switched to db Pravvy

> db.Students.insertMany([

```
{roll:101,name:"Rahul",age:20,marks:85,city:"Lucknow"},
{roll:102,name:"Priya",age:21,marks:90,city:"Kanpur"},
{roll:103,name:"Amit",age:19,marks:75,city:"Varanasi"},
{roll:104,name:"Neha",age:22,marks:88,city:"Prayagraj"},
{roll:105,name:"Rohan",age:20,marks:92,city:"Agra"},
{roll:106,name:"Anjali",age:21,marks:81,city:"Lucknow"},
{roll:107,name:"Vivek",age:23,marks:69,city:"Ayodhya"},
{roll:108,name:"Sneha",age:20,marks:95,city:"Varanasi"},
{roll:109,name:"Karan",age:22,marks:78,city:"Kanpur"},
{roll:110,name:"Pooja",age:19,marks:87,city:"Lucknow"},
{roll:111,name:"Arjun",age:21,marks:91,city:"Agra"},
{roll:112,name:"Simran",age:20,marks:73,city:"Varanasi"},
{roll:113,name:"Aditya",age:22,marks:84,city:"Prayagraj"},
{roll:114,name:"Kavya",age:21,marks:89,city:"Lucknow"},
{roll:115,name:"Ravi",age:23,marks:76,city:"Kanpur"},
{roll:116,name:"Nisha",age:20,marks:93,city:"Agra"},
{roll:117,name:"Sahil",age:19,marks:68,city:"Ayodhya"},
{roll:118,name:"Isha",age:22,marks:96,city:"Varanasi"},
{roll:119,name:"Varun",age:21,marks:82,city:"Lucknow"},
{roll:120,name:"Riya",age:20,marks:86,city:"Kanpur"},
{roll:121,name:"Manish",age:22,marks:74,city:"Agra"},
{roll:122,name:"Ayesha",age:21,marks:90,city:"Varanasi"},
{roll:123,name:"Deepak",age:23,marks:79,city:"Prayagraj"},
{roll:124,name:"Shreya",age:19,marks:94,city:"Lucknow"},
{roll:125,name:"Mohit",age:20,marks:83,city:"Kanpur"},
```

4. DISPLAY ALL STUDENTS

MongoDB Query:

db.students.find()

Explanation: Retrieves documents from the students collection.

```
}
> db.Students.find()
< {
  _id: ObjectId('6ab7c37a21329808c205e3b3'),
  roll: 101,
  name: 'Rahul',
  age: 20,
  marks: 85,
  city: 'Lucknow'
}
{
  _id: ObjectId('6ab7c37a21329808c205e3b4'),
  roll: 102,
  name: 'Priya',
  age: 21,
  marks: 90,
  city: 'Kanpur'
}
{
  _id: ObjectId('6ab7c37a21329808c205e3b5'),
  roll: 103,
  name: 'Amit',
  age: 19,
  marks: 75,
  city: 'Varanasi'
}
```

5. EQUAL TO — \$eq

MongoDB Query:

db.students.find({marks: {\$eq: 85}})

Explanation: Finds students whose marks are exactly 85.

```
> db.Students.find({marks: {$eq: 85}})
< {
  _id: ObjectId('6ab7c37a21329808c205e3b3'),
  roll: 101,
  name: 'Rahul',
  age: 20,
  marks: 85,
  city: 'Lucknow'
}
{
  _id: ObjectId('6ab7c37a21329808c205e3cd'),
  roll: 127,
  name: 'Akash',
  age: 22,
  marks: 85,
  city: 'Varanasi'
}
```

6. GREATER THAN — \$gt

MongoDB Query:

db.students.find({marks: {\$gt: 85}})

Explanation: Finds students whose marks are greater than 85.

```
> db.Students.find({marks: {$gt: 85}})
{
  _id: ObjectId('6ab7c37a21329808c205e3b4'),
  roll: 102,
  name: 'Priya',
  age: 21,
  marks: 90,
  city: 'Kanpur'
}
{
  _id: ObjectId('6ab7c37a21329808c205e3b6'),
  roll: 104,
  name: 'Neha',
  age: 22,
  marks: 88,
  city: 'Prayagraj'
}
```

7. LESS THAN — \$lt

MongoDB Query:

db. students. find ({marks: {\$lt: 70}})

Explanation: Finds students whose marks are less than 70.

```
> db. Students. find ({marks: {$lt: 70}})
< [
  {
    _id: ObjectId('6ab7c37a21329808c205e3b9'),
    roll: 107,
    name: 'Vivek',
    age: 23,
    marks: 69,
    city: 'Ayodhya'
  },
  {
    _id: ObjectId('6ab7c37a21329808c205e3c3'),
    roll: 117,
    name: 'Sahil',
    age: 19,
    marks: 68,
    city: 'Ayodhya'
  }
]
```

8. GREATER THAN OR EQUAL TO — \$gte

MongoDB Query:

db. students. find({marks: {\$gte: 90}})

Explanation: Finds students whose marks are 90 or above.

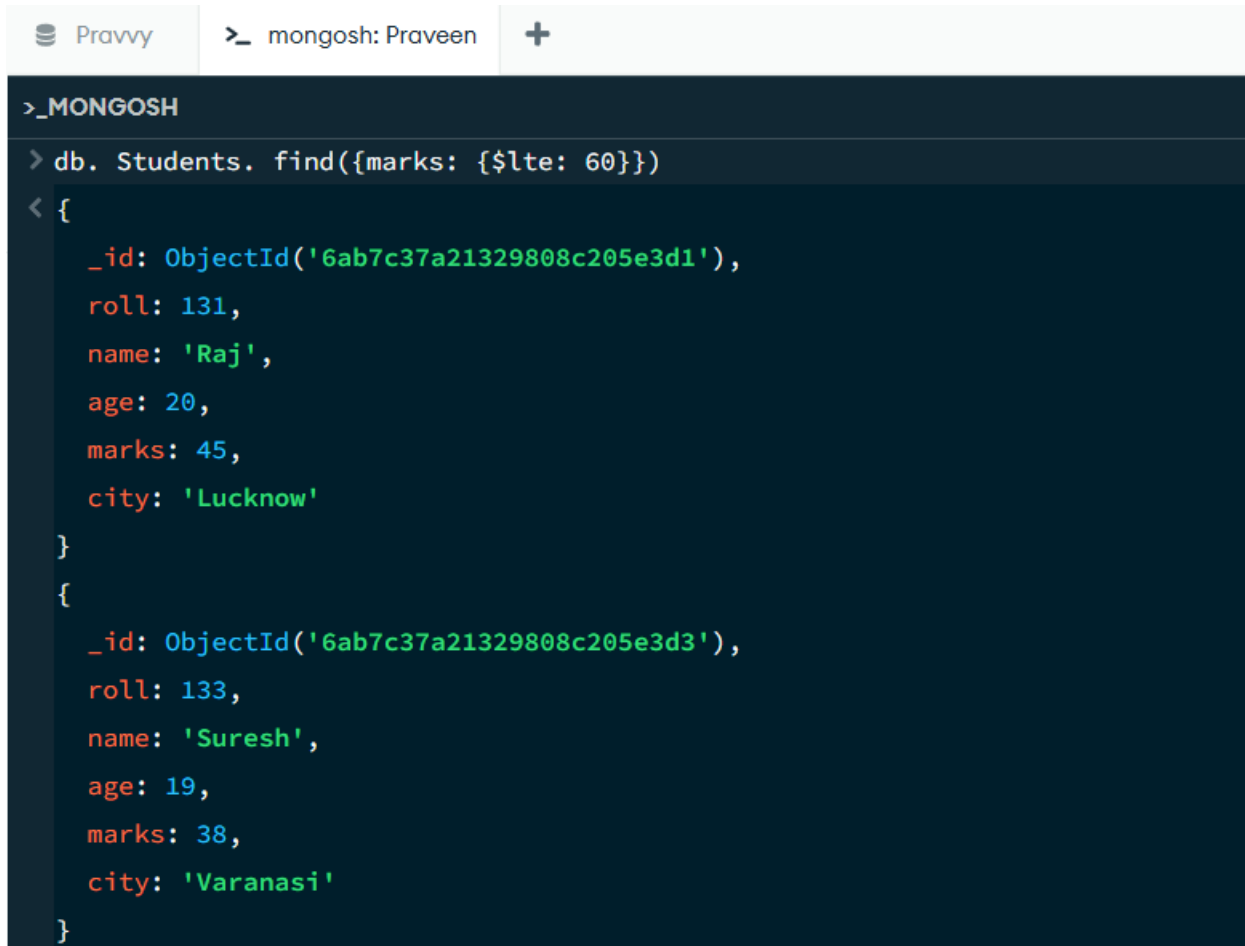
```
> db. Students. find({marks: {$gte: 90}})
< {
  _id: ObjectId('6ab7c37a21329808c205e3b4'),
  roll: 102,
  name: 'Priya',
  age: 21,
  marks: 90,
  city: 'Kanpur'
}
{
  _id: ObjectId('6ab7c37a21329808c205e3b7'),
  roll: 105,
  name: 'Rohan',
  age: 20,
  marks: 92,
  city: 'Agra'
}
```

9. LESS THAN OR EQUAL TO — \$lte

MongoDB Query:

db. students. find({marks: {\$lte: 60}})

Explanation: Finds students whose marks are 60 or below.



The screenshot shows a MongoDB shell interface with a tab labeled 'Pravvy' and a sub-tab 'mongosh: Praveen'. The prompt is '>_MONGOSH'. The command entered is 'db. Students. find({marks: {\$lte: 60}})'. The output is a JSON array of two student documents. The first document has _id: ObjectId('6ab7c37a21329808c205e3d1'), roll: 131, name: 'Raj', age: 20, marks: 45, and city: 'Lucknow'. The second document has _id: ObjectId('6ab7c37a21329808c205e3d3'), roll: 133, name: 'Suresh', age: 19, marks: 38, and city: 'Varanasi'.

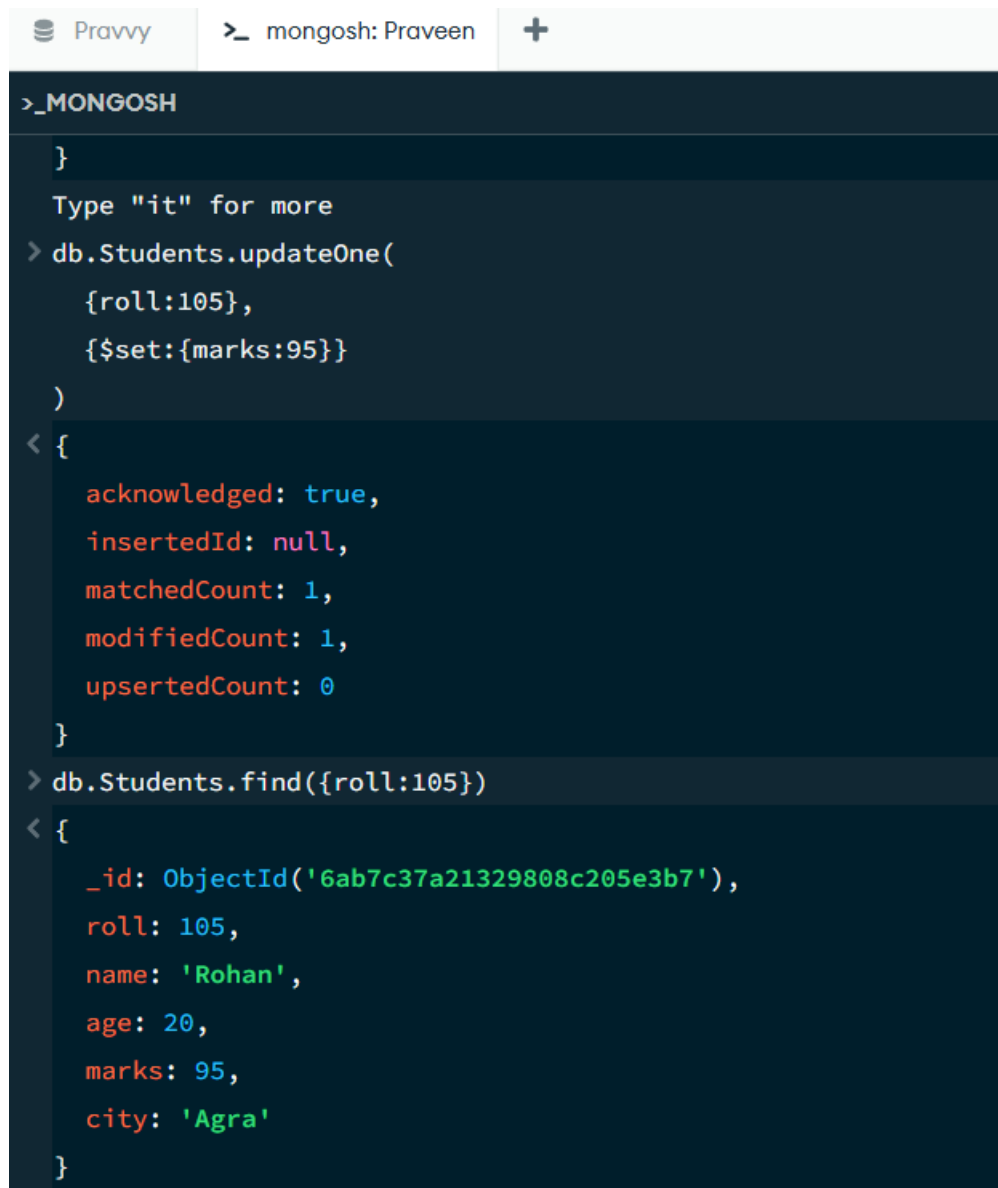
```
>_MONGOSH
> db. Students. find({marks: {$lte: 60}})
< [
  {
    _id: ObjectId('6ab7c37a21329808c205e3d1'),
    roll: 131,
    name: 'Raj',
    age: 20,
    marks: 45,
    city: 'Lucknow'
  },
  {
    _id: ObjectId('6ab7c37a21329808c205e3d3'),
    roll: 133,
    name: 'Suresh',
    age: 19,
    marks: 38,
    city: 'Varanasi'
  }
]
```

10. UPDATE OPERATION

MongoDB Query:

```
db.students.updateOne(  
  {roll:105},  
  {$set:{marks:95}}  
)
```

Explanation: Updates the marks to 95 of the student with roll number 105.



```
Pravvy  > mongosh: Praveen +  
  
> _MONGOSH  
}  
Type "it" for more  
> db.Students.updateOne(  
  {roll:105},  
  {$set:{marks:95}}  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}  
> db.Students.find({roll:105})  
< {  
  _id: ObjectId('6ab7c37a21329808c205e3b7'),  
  roll: 105,  
  name: 'Rohan',  
  age: 20,  
  marks: 95,  
  city: 'Agra'  
}
```

11. DELETE ONE OPERATION

MongoDB Query:

```
db.students.deleteOne({roll:10})
```

Explanation: Deletes one document matching roll number 10.

```
> db.Students.deleteOne({roll:105})
< {
  acknowledged: true,
  deletedCount: 1
}
Pravvy > |
```

12. DELETE MANY OPERATION

MongoDB Query:

```
db.students.deleteMany({
  marks: {$lt:40}
})
```

Explanation: Deletes all students having marks below 40.

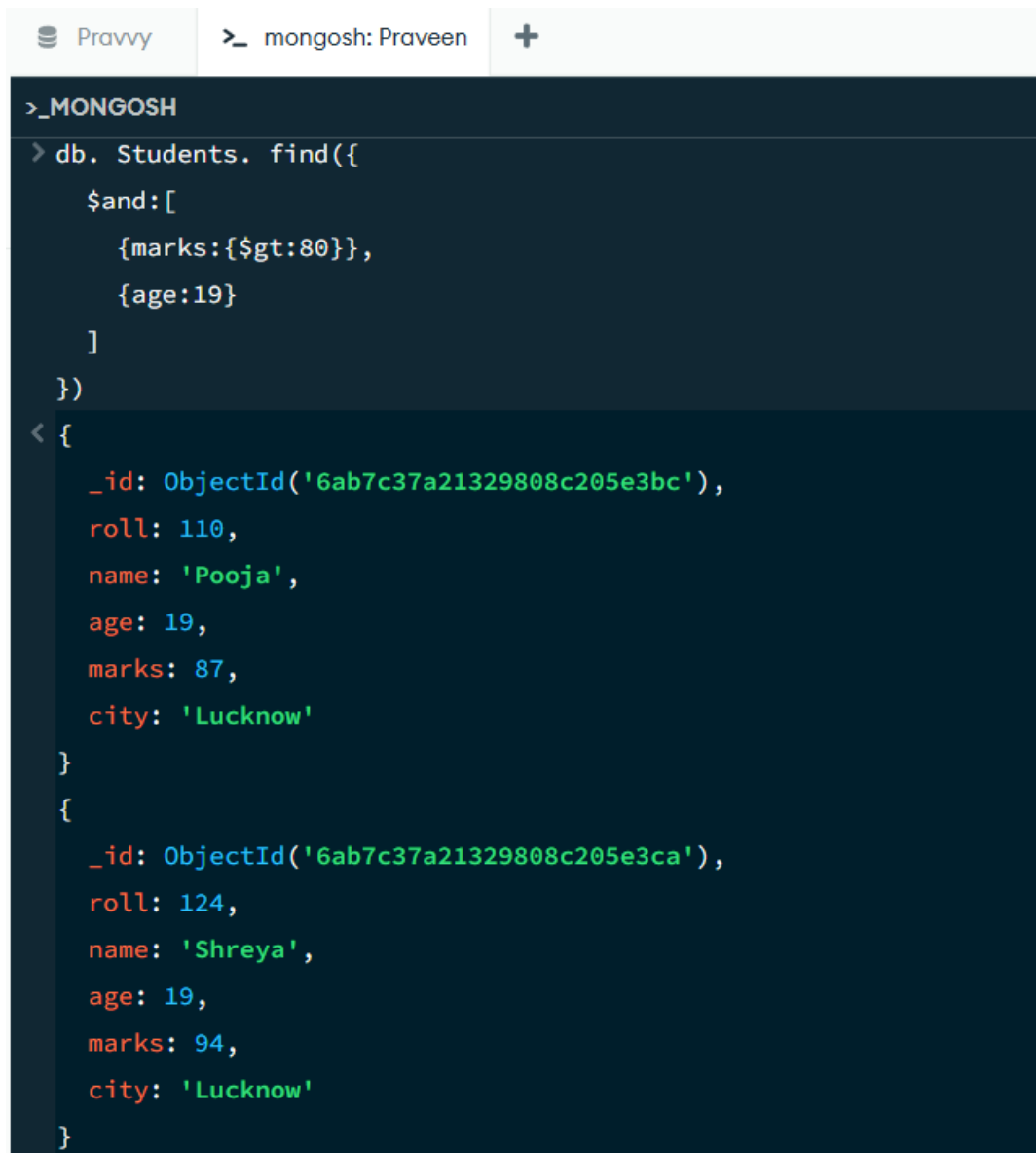
```
> db.Students.deleteMany({
  marks: {$lt:40}
})
< {
  acknowledged: true,
  deletedCount: 9
}
Pravvy > |
```

13. AND OPERATION — \$and

MongoDB Query:

```
db.students.find({
  $and:[
    {marks:{$gt:80}},
    {age:19}
  ]
})
```

Explanation: Finds students satisfying both conditions.



The screenshot shows a MongoDB shell interface with two tabs: 'Pravvy' and 'mongosh: Praveen'. The active tab is 'mongosh: Praveen'. The prompt is '> _MONGOSH'. The query entered is:

```
> db.Students.find({
  $and:[
    {marks:{$gt:80}},
    {age:19}
  ]
})
```

The output shows two documents:

```
< {
  _id: ObjectId('6ab7c37a21329808c205e3bc'),
  roll: 110,
  name: 'Pooja',
  age: 19,
  marks: 87,
  city: 'Lucknow'
}
{
  _id: ObjectId('6ab7c37a21329808c205e3ca'),
  roll: 124,
  name: 'Shreya',
  age: 19,
  marks: 94,
  city: 'Lucknow'
}
```

14. OR OPERATION — \$or

MongoDB Query:

```
db.students.find({
  $or:[
    {city:"Lucknow"},
    {city:"Kanpur"}
  ]
})
```

Explanation: Finds students from Lucknow or Kanpur.



The screenshot shows a MongoDB shell interface with a tab labeled 'Pravvy' and a sub-tab 'mongosh: Praveen'. The prompt is '>_MONGOSH'. The user has entered the query: `> db.Students.find({ $or:[ {city:"Lucknow"}, {city:"Kanpur"} ] })`. The results are displayed as a JSON array with three student objects. Each object contains fields for `_id`, `roll`, `name`, `age`, `marks`, and `city`.

```
>_MONGOSH
> db.Students.find({
  $or:[
    {city:"Lucknow"},
    {city:"Kanpur"}
  ]
})
< [ {
  _id: ObjectId('6ab7c37a21329808c205e3b3'),
  roll: 101,
  name: 'Rahul',
  age: 20,
  marks: 85,
  city: 'Lucknow'
}, {
  _id: ObjectId('6ab7c37a21329808c205e3b4'),
  roll: 102,
  name: 'Priya',
  age: 21,
  marks: 90,
  city: 'Kanpur'
}, {
  _id: ObjectId('6ab7c37a21329808c205e3b8'),
  roll: 106,
  name: 'Anjali',
  age: 21,
  marks: 81,
  city: 'Lucknow'
} ]
```

15. SORT OPERATION

A.MongoDB Query:

db. students. find (). sort({marks : 1})

Explanation: Sorts students by marks from lowest to highest. Use 1 for ascending order.

```
>_MONGOSH
> db.Students.find().sort({marks: 1})
< {
  _id: ObjectId('6ab78a8b6cd8fe8b0f1ceffe'),
  roll: 149,
  name: 'Varsha',
  age: 21,
  marks: 41,
  city: 'Lucknow'
}
{
  _id: ObjectId('6ab78a8b6cd8fe8b0f1ceff5'),
  roll: 140,
  name: 'Mansi',
  age: 19,
  marks: 43,
  city: 'Lucknow'
}
{
  _id: ObjectId('6ab78a8b6cd8fe8b0f1cefec'),
  roll: 131,
  name: 'Raj',
  age: 20,
  marks: 45,
  city: 'Lucknow'
}
{
  _id: ObjectId('6ab78a8b6cd8fe8b0f1ceff9'),
  roll: 144,
```

B. MongoDB Query:

db.students.find().sort({marks: -1})

Explanation: Sorts students by marks from highest to lowest. Use -1 for descending order.

```
>_MONGOSH
> db.Students.find().sort({marks: -1})
< {
  _id: ObjectId('6ab788bf6cd8fe8b0f1cefe9'),
  roll: 128,
  name: 'Muskan',
  age: 20,
  marks: 98,
  city: 'Lucknow'
}
{
  _id: ObjectId('6ab788bf6cd8fe8b0f1cefd1'),
  roll: 118,
  name: 'Isha',
  age: 22,
  marks: 96,
  city: 'Varanasi'
}
{
  _id: ObjectId('6ab788bf6cd8fe8b0f1cefd2'),
  roll: 105,
  name: 'Rohan',
  age: 20,
  marks: 95,
  city: 'Agra'
}
{
```

16. LIMIT OPERATION

MongoDB Query:

db. students. find (). limit (5)

Explanation: Displays only the first five documents.

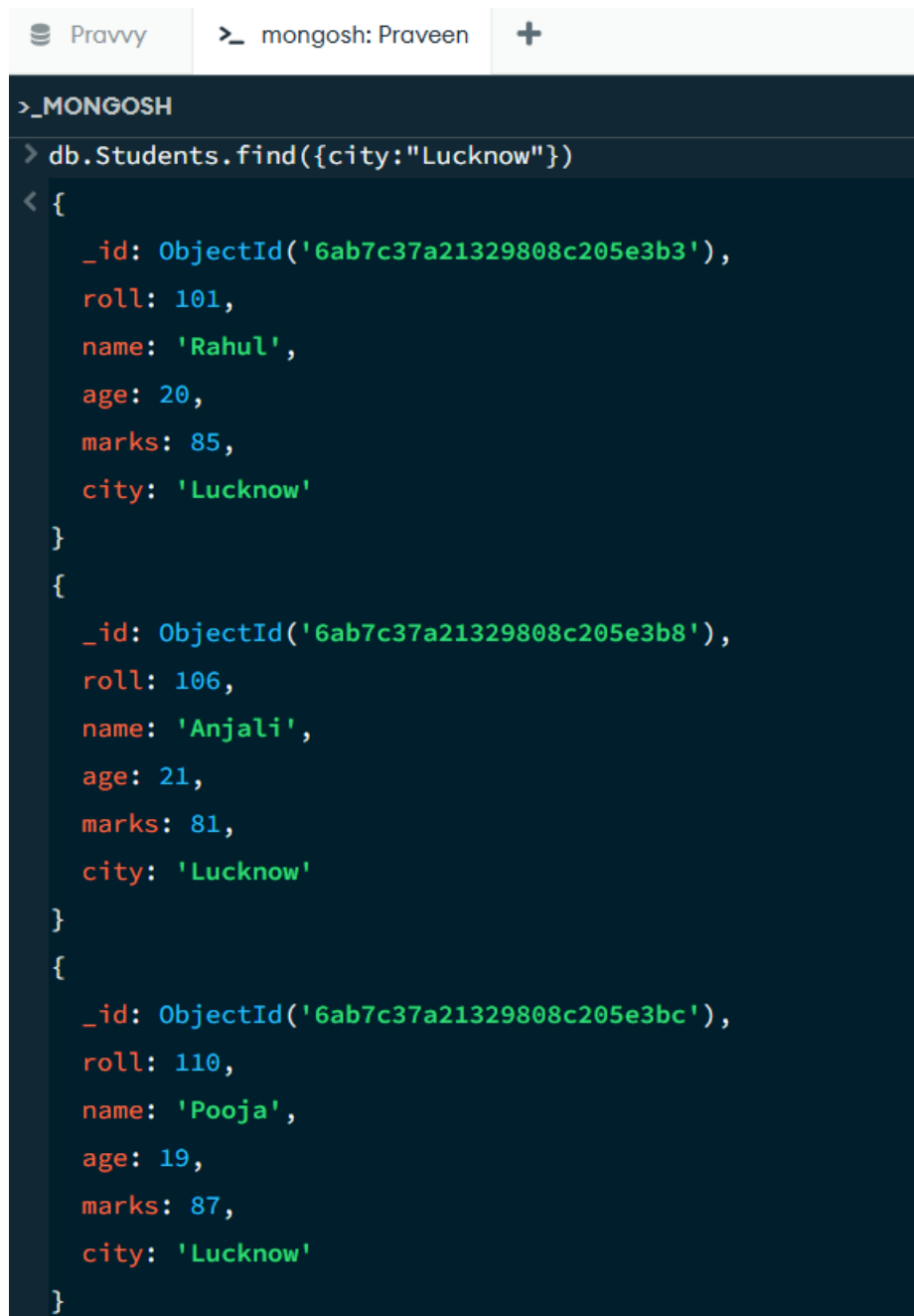
```
> db. Students. find (). limit (5)
< {
  _id: ObjectId('6ab7c37a21329808c205e3b3'),
  roll: 101,
  name: 'Rahul',
  age: 20,
  marks: 85,
  city: 'Lucknow'
}
{
  _id: ObjectId('6ab7c37a21329808c205e3b4'),
  roll: 102,
  name: 'Priya',
  age: 21,
  marks: 90,
  city: 'Kanpur'
}
{
  _id: ObjectId('6ab7c37a21329808c205e3b5'),
  roll: 103,
  name: 'Amit',
  age: 19,
  marks: 75,
  city: 'Varanasi'
}
```

17. SEARCH BY CITY

MongoDB Query:

```
db.students.find({city:"Lucknow"})
```

Explanation: Finds all students whose city is Lucknow.



The screenshot shows a MongoDB shell interface with a tab labeled 'Pravvy' and a prompt 'mongosh: Praveen'. The command prompt is '>_MONGOSH'. The query entered is 'db.Students.find({city:"Lucknow"})'. The output shows three JSON documents representing students from Lucknow.

```
>_MONGOSH
> db.Students.find({city:"Lucknow"})
< {
  _id: ObjectId('6ab7c37a21329808c205e3b3'),
  roll: 101,
  name: 'Rahul',
  age: 20,
  marks: 85,
  city: 'Lucknow'
}
{
  _id: ObjectId('6ab7c37a21329808c205e3b8'),
  roll: 106,
  name: 'Anjali',
  age: 21,
  marks: 81,
  city: 'Lucknow'
}
{
  _id: ObjectId('6ab7c37a21329808c205e3bc'),
  roll: 110,
  name: 'Pooja',
  age: 19,
  marks: 87,
  city: 'Lucknow'
}
```

18. COUNT DOCUMENTS

MongoDB Query:

db. students. count Documents ()

Explanation: Counts documents currently present in the collection (performing after the delete function).

```
> db.Students.countDocuments()
```

```
< 140
```

```
Pravvy >
```